

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №2
по дисциплине «Системы параллельной обработки данных»
Тема: Коллективные операции

Студент гр. 1306

Шаврин А.П.

Преподаватель

Татаринов Ю.С.

Санкт-Петербург

2025

Цель работы.

Изучить, как использовать коллективные операции в библиотеке MPI. Реализовать программу, использующую данную функцию как основную часть для решения задачи. Проанализировать работу программы.

Постановка задачи.

Вариант 1. В каждом процессе дано вещественное число. Используя функцию MPI_Gather, переслать эти числа в главный процесс и вывести их в порядке возрастания рангов переславших их процессов (первым вывести число, данное в главном процессе).

Выполнение работы.

Реализация началась с инициализации MPI в функции main: вызов MPI_Init для запуска среды, получение ранга (MPI_Comm_rank) и общего числа процессов (MPI_Comm_size).

Затем была вынесена основная логика в функцию processTask, чтобы избежать дублирования кода и упростить отладку: каждый процесс генерирует случайное число с помощью std::srand (инициализировано временем + рангом) и std::rand, нормализуя его к диапазону 0–100 (делением на 10.0). Использование ранга необходима для уникальной генерации на каждом процессе.

Далее следует вызов complexComputation — пустой цикл для искусственной вычислительной нагрузки. Это было сделано для анализа масштабируемости по условиям лабораторной для исследования ускорения/замедления программы по числу процессов и сложности задачи. После чего следует вывод сгенерированного процессом числа.

Затем каждый процесс вызывает MPI_Gather: отправка одного double от каждого процесса в буфер root-процесса (вектор all_numbers размером size, инициализированный только на 0 процессе). Это коллективная операция, блокирующая все процессы до завершения сбора, что гарантирует синхронизацию. В root-процессе следует цикл вывода: "Процесс i: значение", в

порядке рангов (i от 0 до $size-1$). После этого следует вызов функции — `MPI_Finalize` и завершение программы.

Для визуального представления алгоритма нарисована сеть Петри для n процессов представленная на рисунке 1. Начальными для каждого процесса являются состояния готовности к генерации, конечными являются состояния, когда каждый процесс отправил свое число в главный процесс, а для первого когда он собрал отправленные числа от всех других и вывел их в консоль.

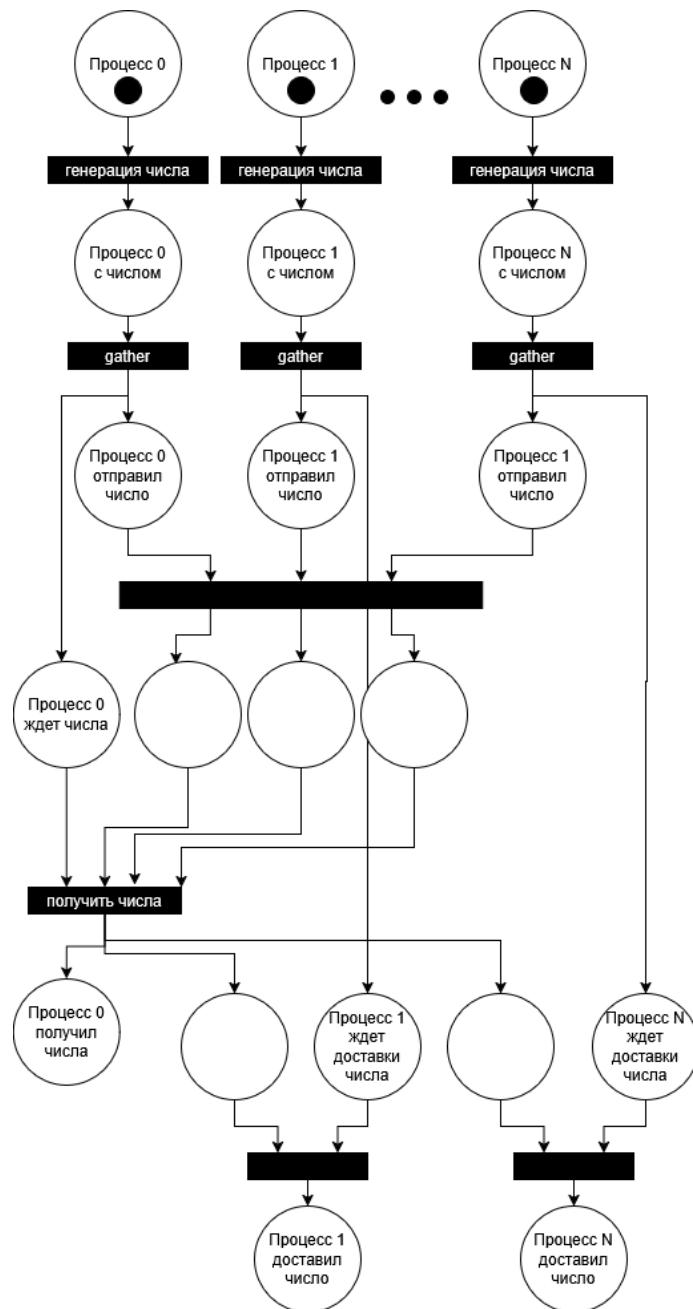


Рисунок 1 – Сеть Петри для n процессов.

Исследуем время выполнения программы при различном размере задачи для различного числа процессов. Результаты экспериментов представлены в таблице 1. Для каждого числа процессов и размеров поля значения приведены в усреднение на 50 запусков программы (время работы программы считалось по среднему времени завершения всех процессов).

Таблица 1.

Сложность задачи	Число процессов										
	2	3	4	5	6	8	10	12	16	24	32
0	0.24	0.49	0.67	0.93	1.75	1.53	2.32	3.39	2.37	3.7	6.36
10 000	0.24	0.48	0.85	1.04	1.48	2.36	2.65	3.14	4.19	6.03	7.46
20 000	0.47	0.91	1.2	1.23	1.84	2.65	2.78	3.08	3.56	4.52	6.48
30 000	0.43	0.57	1.49	1.54	1.8	2.18	2.87	3.59	3.56	6.39	7.85

По данным из таблицы можно сделать несколько ключевых выводов о поведении программы.

Во-первых, из рисунка 2 видно, что время выполнения растет с увеличением числа процессов, причем этот рост становится особенно заметным после 6 процессов. Это напрямую связано с архитектурой процессора - 6 физических ядер не могут эффективно обслуживать большее количество процессов, что приводит к оверсубскрипции и резкому росту накладных расходов. Паттерн роста времени похож на линейный из-за природы операции MPI_Gather, которая создает последовательное узкое место - главный процесс должен получить данные от всех остальных, что занимает время, пропорциональное числу процессов. Синхронизация в коллективных операциях заставляет быстрые процессы ждать медленные, усугубляя проблему. Можно видеть также, что сложность задачи слабо влияет на общую форму кривых, поскольку накладные расходы MPI доминируют над вычислительной составляющей.

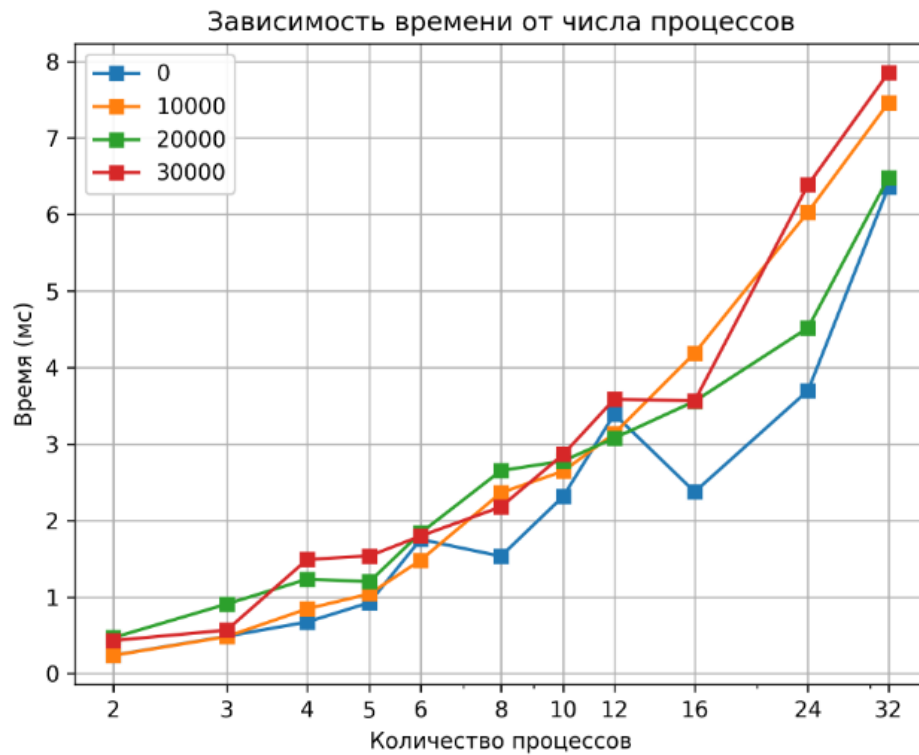


Рисунок 2. Зависимость времени от числа процессов.

Во-вторых, из рисунка 3 видно, данные демонстрируют закономерность: при увеличении сложности задачи время выполнения растет незначительно, в то время как рост числа процессов оказывает гораздо более существенное влияние.

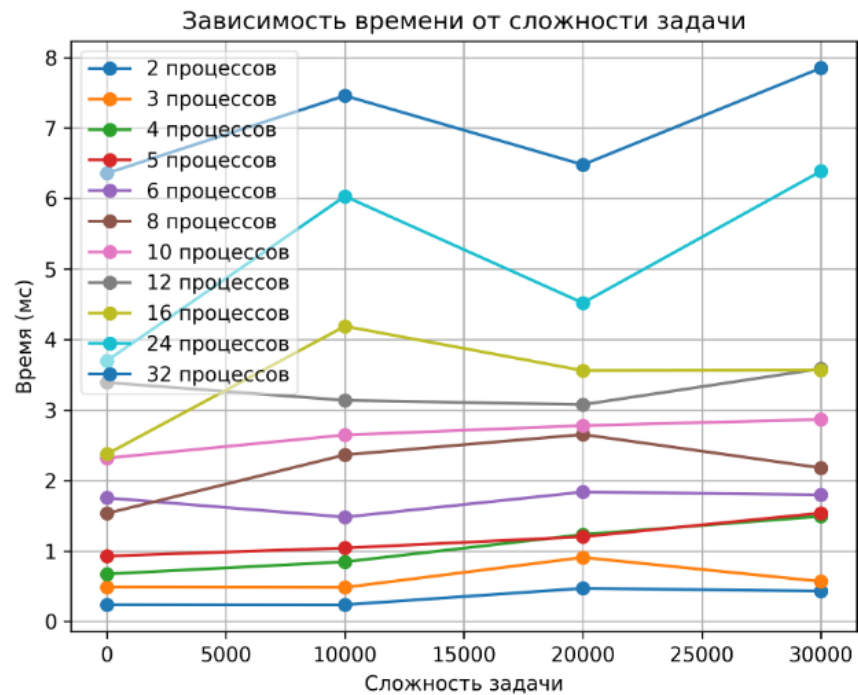


Рисунок 3. Зависимость времени от сложности задачи.

Рассчитаем ускорение программы по формуле:

$$S_p(n) = \frac{T_1(n)}{T_p(n)}, \text{ где } p - \text{число процессов, а } n - \text{размеры поля}$$

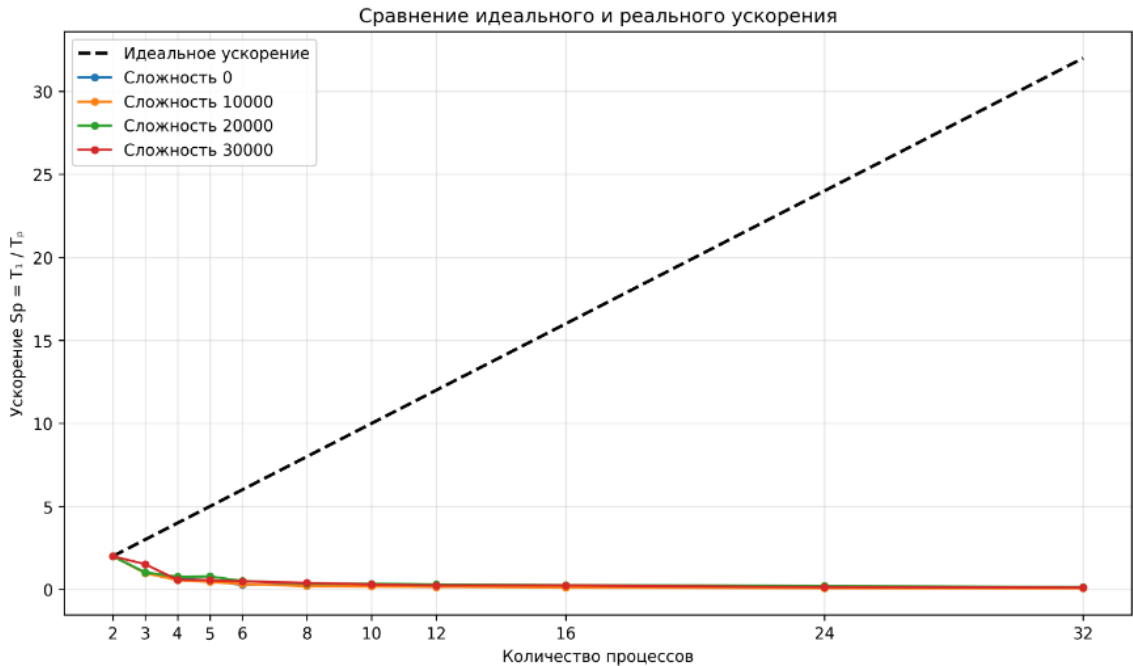


Рисунок 4. Ускорение программы для разной сложности задачи при увеличении числа процессов.

Как видно из рисунка 4, данные демонстрируют отсутствие реального ускорения при увеличении числа процессов. Каждый процесс независимо выполняет свой цикл. Однако эффект от этого распараллеливания полностью теряется из-за операции `MPI_Gather`, которая работает как синхронизационный барьер. Видно, что ускорение падает до 6 процессов, а потом почти равно нулю. Это связано с тем, что на машине, на которой проводились тесты, всего 6 ядер. При превышении данного числа процессы делят ядра через быстрое переключение контекста, когда планировщик ОС поочередно выделяет им кванты времени на одних и тех же физических ядрах. Соответственно время выполнения начинает сильно расти из-за накладных расходов, а ускорение близится к 0.

Исходный код программы находится в приложении А, результат выполнения программы для 4х процессов находится в приложении Б.

Выводы.

По итогам лабораторной работы реализована программа на C++ с использованием библиотеки MPI, демонстрирующая коллективную операцию MPI_Gather для сбора вещественных чисел, сгенерированных в каждом процессе, в главный процесс и их последующий вывод в порядке возрастания рангов (начиная с числа главного процесса). Для анализа масштабируемости введена искусственная вычислительная нагрузка через цикл, что позволило исследовать зависимость времени выполнения от числа процессов и сложности задачи.

Из полученных результатов видно, что время выполнения растёт с увеличением числа процессов, особенно после 6 (количества физических ядер), что обусловлено конкуренцией за ядра и накладными расходами MPI_Gather как последовательного узкого места с синхронизацией всех процессов. При этом ускорение не достигается ($S \leq 1$), а падает до нуля на больших количествах процессов, поскольку отсутствует распределение нагрузки — каждый процесс выполняет полный объём вычислений независимо, а сбор данных создаёт барьер ожидания.

Аналогично, при росте сложности задачи (n) время увеличивается линейно, но незначительно, так как вычисления доминируют только на малых числах процессов, а на больших коммуникации перевешивают. Отклонения в данных (например, ломаные кривые) объясняются вариациями выделения ресурсов ОС и шумом (прерывания, кэширование), несмотря на усреднение по 50 запускам с медианой; при этом сохраняется общая тенденция нелинейного роста времени от числа процессов и слабой линейности от сложности задачи, подчёркивающая пределы эффективности параллелизма без балансировки нагрузки.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
    sea_battle.cpp
#include <iostream>
#include <vector>
#include <mpi.h>
#include <cstdlib>
#include <ctime>
#include <fstream>
#include <numeric>
#include <algorithm>

#define COMPLEXITY 30000

void complexComputation(int iterations) {
    long long counter = 0;
    for (int i = 0; i < iterations; i++) {
        counter++;
    }
}

void processTask(int rank, int size) {
    std::srand(std::time(nullptr) + rank);
    double random_number = (std::rand() % 1000) / 10.0;

    complexComputation(COMPLEXITY);

    std::cout << "Процесс " << rank << ": мое число = " << random_number
    << std::endl;

    std::vector<double> all_numbers;
    if (rank == 0) {
        all_numbers.resize(size);
    }

    MPI_Gather(&random_number, 1, MPI_DOUBLE,
              all_numbers.data(), 1, MPI_DOUBLE,
              0, MPI_COMM_WORLD);

    if (rank == 0) {
        std::cout << "\n=== РЕЗУЛЬТАТ ===" << std::endl;
        std::cout << "Числа в порядке возрастания рангов процессов:" <<
        std::endl;

        for (int i = 0; i < size; i++) {
            std::cout << "Процесс " << i << ": " << all_numbers[i] <<
            std::endl;
        }
    }
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
```

```

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

double startTime = MPI_Wtime();
processTask(rank, size);
double endTime = MPI_Wtime();
double processTime = (endTime - startTime) * 1000;

std::vector<double> allTimes;
if (rank == 0) {
    allTimes.resize(size);
}
MPI_Gather(&processTime, 1, MPI_DOUBLE,
          allTimes.data(), 1, MPI_DOUBLE,
          0, MPI_COMM_WORLD);

if (rank == 0) {
    double maxTime = *std::max_element(allTimes.begin(),
allTimes.end());
    double avgTime = std::accumulate(allTimes.begin(),
allTimes.end(), 0.0) / size;

    std::cout << "\n=== РЕЗУЛЬТАТЫ ДЛЯ " << size << " ПРОЦЕССОВ ==="
<< std::endl;
    std::cout << "Максимальное время: " << maxTime << " мс" <<
std::endl;
    std::cout << "Среднее время: " << avgTime << " мс" << std::endl;

    std::ofstream file("results_30000.txt", std::ios::app);
    if (file.is_open()) {
        std::cout << "open" << std::endl;
        file << size << " " << maxTime << " " << avgTime <<
std::endl;
        file.close();
    }
}
MPI_Finalize();

return 0;
}

```

ПРИЛОЖЕНИЕ Б

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

Процесс 2: мое число = 59.6
Процесс 3: мое число = 39.5
Процесс 0: мое число = 86.9
Процесс 1: мое число = 11.6

=== РЕЗУЛЬТАТ ===

Числа в порядке возрастания рангов процессов:

Процесс 0: 86.9
Процесс 1: 11.6
Процесс 2: 59.6
Процесс 3: 39.5